Sanbao OpenSDK Documentation

Author	Date	Version	Modify/Add/Delete
Cris_Peng	2019/11/6	2.0.1.10	1. Added robot type query interface (3.7.10)
Cris_Peng	2019/10/28	2.0.1.10	1. Change of control protocol for the Little Elf robot arm: changed from HandMotionManager to WingMotionManager control, and the corresponding motion control class has been changed. 1. RelativeAngleWingMotion, AbsoluteWingHandMotion, NoAngleWingMotion (corresponding protocol updates, such as 3.5 Wing Motion Control) 2. Modification of the Phantom Wheel Angle-Free Control Protocol: See 3.6.1 for details; mainly field modifications. 3. Compatibility with the D-type head motor control protocol. For D-type head motion control, please use DRelativeAngleHeadMotion, DAbsoluteAngleHeadMotion, and DNoAngleHeadMotion (the usage of these three classes is consistent with the D-type SDK documentation, only the class names have changed. For Phantom version head control, please see 3.4 of this document). Add the D-type robot control methods doDNoAngleMotion, doDAbsoluteAngleMotion, and doDRelativeAngleMotion to HeadMotionMa
Cris_Peng	2019/6/3	2.0.1.8	1. Added Desktop Motion Control Protocol (3.15)
Cris_Peng	2019/3/7	2.0.1.7	1. Client heartbeat is maintained in a background thread to avoid being blocked by the main thread. 2. Added desktop robot face tracking switch and status callback. ÿ3.14ÿ
Cris_Peng	2018/10/26	2.0.1.4	1. Added wheel motion callback (see 3.6.4 for details) 2. Added gyroscope data callback
Cris_Peng	2018/10/10	2.0.1.3	1. Added language switching recognition (3.2.16 only applicable to international versions) 2. Added disabling semantic request pre-configuration (3.11.9 only applicable to international versions) 3. Updated methods related to pulling video streams, removing the MediaManager management class and replacing it with...

			HDCameraManager 1.
Cris_Peng 2018/9/12 2.0.1.2			Add anti-false positive configuration (3.11.8) 2. Method to hide system floating button in App (3.7.9)
Cris_Peng 2018/8/3		2.0.1.1 1. Was the wake-up interface added via voice activation or for some other reason?	For changes to the wake-up process (touch, calling the SpeechManager.doWakeUp method), please see [link/reference]. 3.2.5ÿ 2. Added a false recognition mode. Currently, the main controller will actively block and filter the recognition of one character. After setting the false recognition mode, the main controller will no longer block the recognition of one character (see 3.11.7 False Recognition Mode). 3. Added an interface to cancel the last recognition (currently for internal use only). This update is only valid on system versions released on or after August 3, 2018.
Cris_Peng 2018/6/21 2.0.1.0		The recognition interface returns "Recognition Start", "Recognition End", and "Recognition Error".	Error returning an error (only applies to the domestic version). For details, please see versions 3.2.13, 3.2.14, and 3.2.15.

Table of contents

SDK Version Change Notice

Version number 2.0.1.10:

1. Added robot type query interface (3.7.10)

Version 2.0.1.9:

1. Change to the control protocol of the Little Elf robot arm: changed from the original HandMotionManager to...

The WingMotionManager control should be used, and the corresponding motion control class should be changed to...

1. RelativeAngleWingMotion, AbsoluteWingHandMotion, NoAngleWingMotion (corresponding protocol updates, such as 3.5 Wing Motion Control) 2. Modification of the Phantom Wheel

Angle-Free Control Protocol: See 3.6.1 for details. The main modification is the constant modification of the motion mode. 3. Compatibility with the D-type head motor control protocol. For D-type head motion control, please use DRelativeAngleHeadMotion, DAbsoluteAngleHeadMotion, and DNoAngleHeadMotion (the usage of these three classes is consistent with the usage in the D-type SDK documentation, only the class names have changed. For Phantom version head control, please see 3.4 of this document). Add the D-type robot control methods doDNoAngleMotion, doDAbsoluteAngleMotion, and doDRelativeAngleMotion to HeadMotionManager.

Version 2.0.1.8:

1. Add desktop robot control protocol (3.15)

Version 2.0.1.7:

1. Client heartbeat is maintained in a background thread to avoid being blocked by the main thread. 2. Added a face tracking switch and status callback for the desktop robot.

Version 2.0.1.4:

1. Add wheel motion callback 2. Add gyroscope data callback

Version 2.0.1.3:

1. Added language switching recognition (3.2.16 only applicable to international versions) 2. Added disabling semantic request pre-configuration (3.11.9 only applicable to international versions) 3. Updated methods related to pulling video streams, removing the MediaManager management class and replacing it with... HDCameraManager

Version 2.0.1.2:

1. Background music playback anti-virus configuration. 2. Added another floating button hiding function; it will no longer flash briefly when switching between activities within the same app.

Version 2.0.1.1:

1. Add a wake-up interface that specifies whether wake-up is via voice or other reasons (touch, calling the SpeechManager.doWakeUp method). 2. Add a false recognition mode. Currently, the main controller actively blocks and filters the recognition of one character. After setting the false recognition mode, the main controller will no longer block the recognition of one character. 3. Add an interface to cancel the last recognition (currently for internal use only).

Version number 2.0.1.0:

1. The SDK package name has been changed to com.sanbot.opensdk and the SDK name has been changed to SanbotOpenSDK.

2. Added interfaces for recognition start, recognition end, and recognition error return (see 3.2.13, 3.2.14, and 3.2.15 for details).

Version number 2.0.0:

3. The core communication mechanism of the SDK has been modified, and the SDK's programming language has been changed.

4. The communication mechanism of the projector has been modified, and some of the projector's interfaces have been adjusted.

5. Added obstacle detection interface.

6. Now, **the `register(Class subClass)` method** also needs to be called in the `onCreate` method of the Activity.

The method is to register the Activity's Class object with the SDK.

Version 1.1.8:

1. An interface for acquiring gyroscope sensor data has been added.

2. The relative and absolute angular movements of hand motion control increase the ability to control both hands simultaneously.

Law.

3. Added the ability to customize global command terms and semantics.

4. Corrected the description of the touch event callback section and the description of the wheel's motion without angle.

5. The voice recognition callback has added open semantic functionality.

6. The face recognition callback interface will now return all recognizable face data.

7. Fixed the issue of projector interface failure.

8. Fixed a bug where face recognition would still trigger callbacks after the Activity was destroyed.

9. Fixed a bug that could cause ANR when switching Activity pages while sending commands at high frequency.

10. Fixed a bug in the SDK that could cause the controller to crash.

11. Added an interface for showing/hiding the system FloatBar.

Version 1.1.7:

1. A modular motion manager has been added, which allows control over free walking and automatic charging functions.

2. Added preprocessing command functionality, integrating all previous recording, voice mode, and truncation function configurations into a unified system.

The preprocessing directive has been executed.

3. The power manager has been removed, and the interface for obtaining robot power and charging status has been moved to the system manager.

4. Now, the Service needs to call **the `register(Class subClass)` method** in the `onCreate` method to...

The Service's Class object is registered in the SDK.

5. Added pause, resume, and stop voice synthesis functions; retrieve main control version number; retrieve infrared ranging results; and enable/disable the black line crossing.

Filter, set the brightness of the white light, and monitor the security alarm interface.

6. To ensure compatibility with future semantic functions, the interface for recognizing utterance callbacks has been changed.

1 Overview

This document is intended for Android developers.

This document guides developers on how to quickly integrate the Sanbao Open SDK, which provides Android applications with functions for controlling robot voice, movement, and other related features.

The document describes the robot names as follows: Model B = Little Elf; Model D = King Kong; Desktop Robot = Mini Little Elf.

2. Access Process

2.1 Development Environment

This SDK can be developed using development tools such as Android Studio or Eclipse. It is recommended that Android Studio version 1.5 or higher be used.

The JDK version required is JDK 7 or above, and the minimum minSdkVersion of the Android SDK is 11.

2.2 Configuration Instructions

To accommodate development needs in different environments, we provide three different SDK versions: SanbotOpenSDK_XXX.aar, JarA, and JarB.

The difference between JarA and JarB is that JarB does not provide BindBaseActivity, TopBindActivity, and... The base classes BindBaseService require the BindBaseUtil class to communicate with the robot.

2.2.1 AAR SDK Configuration Instructions

First, please download the provided SanbotOpenSdk_XXX.aar (XXX represents the version number) and gson-2.2.4.jar. Copy the file to the libs folder of the Module.

Add the following content to the build.gradle folder of the App Module:

```
repositories { flatDir { dirs
                'libs'

                }
}
dependencies
    { compile(name:'SanbotOpenSdk_XXX', ext:'aar')
}
```

2.2.2 JAR SDK Configuration

Instructions 1. Please place the JAR folder (or JAR folder, depending on whether you need to use JAR version A or version B) in the JAR folder. Copy all files from the src/libs directory of the JAR file to the libs folder in your project.

2. Please merge the resource files in the src/res directory of the JarA (or JarB) folder with the resource files in your project.

3. Please add the following two permissions to your project's AndroidManifest.xml file:

```
<uses-permission
android:name="android.permission.CAMERA" /> <uses-permission

android:name="android.permission.INTERNET"/> <uses-permission

android:name="android.permission.ACCESS_WIFI_STATE"/> <uses-permission

android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

4. If you are using Android Studio for development, please remember to configure the path for reading .so files by adding the following content to the build.gradle folder of your module:

```
sourceSets.main {
        jniLibs.srcDir 'libs'
}
```

2.3 Explanation of Confusion

When packaging and obfuscating applications that integrate the Sanbao SDK, care must be taken to ensure that methods related to the Sanbao SDK are not obfuscated. The obfuscation method is as follows:

```
-keep class com.qihancloud.opensdk.** {*;} -keep class
com.sanbot.opensdk.**{*;} -keep class
com.sunbo.main.**{*;}
```

Ensure that the classes in the SDK are not obfuscated; otherwise, runtime exceptions such as robot control failures may occur.

2.4 Precautions

1. Currently, the development process requires the use of our provided R&D system for testing. The official system will only support the existing functions in version 2.21 or later.

2. The system does not include the Google Services Framework, meaning all Google-provided services, such as Google Maps and Google Sign-in, are unavailable. Developers are advised to avoid using the corresponding SDKs from the Google Android Open Platform. 3.

Please be cautious when listening to the system boot broadcast. The system boot broadcast's sending time has been changed to the first time the user enters the system desktop after booting. Before using this, please carefully consider whether it will affect your

application. 4. The robot will automatically return to the desktop after the user has stopped operating it for a period of time. If you do not want your app to be returned to the desktop while open, please have your app call the Android system SDK to enable the always-on screen function. Reference code is as follows:

```
getWindow().setFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON,
WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);
```

3. Detailed introduction to encoding access

3.1 Basic Description of the SDK

The SDK only supports controlling the bot within Activity and Service components.

3.1.1 Explanation of Activity Inheritance Classes

All Activity classes in the project should inherit from either the BindBaseActivity or TopBaseActivity class, where TopBaseActivity is a subclass of BindBaseActivity. The differences and similarities between BindBaseActivity and TopBaseActivity are as follows:

Similarities: Both require overriding an abstract method `onMainServiceConnected()` (which is only created in the Activity). (Called once).

Differences: Pages inheriting from the TopBaseActivity class will display the system's default status bar at the top, and the layout can be set. When using this method, please use the setBodyView method. The Application style must be set to the NoActionBar style.

Note: If you want the robot to be controlled immediately upon Activity launch, the corresponding control code must be placed in `onMainServiceConnected()`. Otherwise, placing it in other lifecycle methods will be ineffective! Furthermore, you need to register the `XXActivity` using `register(XXActivity.Class)` in the `onCreate` method. The `register` method must be called before `super.onCreate()`.

3.1.2 Description of Service Inheritance Class

If you need to control the robot within a Service, you can inherit the BindBaseService class. Inheriting BindBaseService requires overriding an abstract method onMainServiceConnected() (which is only called once when the Service is created), and **registering the XXService using register(XXService.Class) in the onCreate method. The register method must be called before super.onCreate(). If you need to control the robot from the moment the Service starts, the corresponding** control code must be in onMainServiceConnected(); otherwise, it will be ineffective.

Example code:

```
public class SampleService extends BindBaseService {
……
 @Override
 protected void onMainServiceConnected() {
 control code
 }
    ……
@Override
    public void onCreate()
            { register(SampleService.class);
      super.onCreate(); }

……
}
```

3.1.3 Special Notes for Jar Package

B Jar package B is an SDK provided to solve the problem of how to control the robot when developers cannot inherit from BindBaseActivity and BindBaseService. In Jar package B, the BindBaseUtil class is used instead, and this class contains the following methods that are required:

1ÿBindBaseUtilÿActivity activityÿ ÿ **BindBaseUtilÿService serviceÿ**

The above two methods are the constructors of BindBaseUtil.

2ÿpublic void connectService()

Calling this method initiates the connection establishment with the robot's master controller. Communication with the robot can only begin after a successful connection is established. This method needs to be called in the `onResume()` lifecycle method of an Activity or the `onCreate()` lifecycle method of a Service.

3ÿpublic void breakConnection()

Call this method to disconnect from the robot's main controller. This method needs to be called within the Activity's onStop() lifecycle hook. It can be called within a method or within the onDestroy() lifecycle method of a Service.

4ÿpublic void setOnConnectedListener(OnConnectedListener onConnectedListener)

This method listens to whether a connection has been established with the main controller. It checks if a connection to the main controller has been established before calling **the `connectService()` method.** When establishing a connection, this connection process is asynchronous, and this method will be called back after the connection is successfully established. If you need to perform certain operations immediately after successfully establishing a connection with the controller, you can put the relevant code in this callback.

Please note: strictly adhere to the above requirements when placing `connectService()` and `breakConnection()` in the corresponding component's lifecycle methods. Remember to call the `breakConnection()` method, and never call `connectService()` or `breakConnection()` in lifecycle methods that do not meet the requirements!

5ÿpublic void register(Class subClass)

Please call this method in the `oncreate()` method of the Service and Activity components to register the class object of the current Service or Activity. This method must be called before the `super.onCreate()` method. If the class is not registered or the registered object is incorrect, the SDK's preprocessing directives may not work correctly.

3.1.3 Description of the OperationResult Return Value All methods for

controlling the robot will return a parameter of type **OperationResult .** This parameter is used to provide feedback on the result of the method execution: whether the execution was successful, the reason for the failure, and the return value if the execution is successful.

OperationResult has the following three methods:

public int getErrorCode(): The result of the instruction execution. 1 indicates success, and other values less than 0 indicate failure.

public String getDescription(): A description of the instruction execution result. If the instruction execution fails, it will provide a detailed explanation of the reason for the failure.

public String getResult(): An additional field, typically used to store feedback data after the command is executed.

3.2 Voice Control

Access process:

1. Request a SpeechManager object. The request code is as follows:

2. Using the obtained speechManager object, call the corresponding methods to control it.

3.2.1 Speech synthesis

method signature:

public OperationResult startSpeak(String text)

Method description:

The speech synthesis method is used to control a robot to speak specified text content. The language of the synthesized speech is related to the current system language settings.

Example code:

```
speechManager.startSpeak(“This is a test text”);
```

3.2.2 Speech synthesis (specifying synthesis parameters)

method signature:

public OperationResult startSpeak(String text,SpeakOption speakOption)

Method description:

The overloaded method of speech synthesis allows you to specify the language, speech rate, and pitch of the synthesized speech.

Parameter description:

SpeakOption has the following parameters:

`languageType` indicates the language of the synthesized speech, with values ranging from LAG_CHINESE (Chinese).

), LAG_ENGLISH_US (English_United States).

speed represents the speech rate of the synthesized speech, with a value ranging from 0 to 100.

Intonation represents the tone of the synthesized speech, with a value ranging from 0 to 100.

Example code:

```
SpeakOption speakOption = new SpeakOption();
speakOption.setSpeed(70);
OperationResult operationResult = speechManager.startSpeak("This is a test
statement", speakOption);
```

3.2.3 Sleep

method signature:

public OperationResult doSleep()

Method description:

Put the machine into sleep mode and stop it from speaking.

Example code:

```
speechManager.doSleep();
```

3.2.4 Wake-up

method signature:

public OperationResult doWakeUp()

Method description:

This method actively wakes the machine up; it can only be called in an Activity, and will not work if called in a Service.

Example code:

```
speechManager.doWakeUp();
```

3.2.5 Signature of the wake-up callback **method:**

public void setOnSpeechListener(SpeechListener speechListener)

Method description:

`setOnSpeechListener` is a general callback interface for voice control. The parameters passed determine the type of callback function. This callback method is triggered when the robot enters a sleep or wake-up state.

Example code:

```
speechManager.setOnSpeechListener(new WakenListener() {
    @Override
    public void onWakeUp() {
        //Wake-up event callback
    }

    @Override
    public void onSleep() {
        // Sleep event callback
    }

    @Override
    public void onWakeUpStatus(boolean b) {
        // If the wake-up event callback b==true, it means the event was woken up by a wake word.
    }
});
```

3.2.6 Identify **the signature of the**

utterance callback method:

public void setOnSpeechListener(SpeechListener speechListener)

Method description:

Please note: This interface has been changed from version 1.1.7 onwards to ensure compatibility with subsequent semantic features.

This interface provides third-party applications with all text recognized by the robot, excluding the wake word. During this process, the third-party application can decide whether to enable the truncation function (the truncation function prevents the robot from processing or responding to the recognized text). Please note that the robot will not recognize user speech when it is in sleep mode.

By default, the truncation feature is disabled. To enable truncation for the current activity, please...
Add the following preprocessor directive to the AndroidManifest.xml file (see section 3.11 for an explanation of preprocessor directives):

<meta-data android:name="RECOGNIZE_MODE" android:value="1"/>

Note: The execution time of the code in the onRecognizeResult callback method should not exceed 500ms, otherwise it will affect...
To avoid disrupting the normal operation of the entire system, it is not recommended to perform truncation operations in the Service.

Parameter description:

The robot's speech recognition callback implements the RecognizeListener interface, which requires the following methods to be implemented:

boolean onRecognizeResult(Grammar grammar);

Users can obtain the text recognized by the robot using the grammar.getText() method.

The `onRecognizeResult` method returns a boolean value. This return value is determined by the developers based on actual business needs. Please configure the robot so that when the return value is true, it means that the robot will no longer respond to the text, and false, otherwise it will.

The Grammar object contains the content of the user's speech as recognized by the robot, as well as the processing of the user's speech. The result after semantic parsing is currently only supported by systems using the iFlytek Semantic Engine. For an explanation of the Grammar class, please refer to the "Sanbao Open Semantic Specification Document".

Example code:

```
speechManager.setOnSpeechListener(new RecognizeListener() {
 @Override
 public boolean onRecognizeResult(Grammar grammar) {
        System.out.print("The robot recognized the text as"+
grammar.getText());
```

```
                return true;
            }
        });
```

3.2.7 Voice volume callback

method signature:

public void setOnSpeechListener(SpeechListener speechListener)

Method description:

This method is called back when the robot detects sounds in the surrounding environment, returning the volume of the detected sound. This method can be used to determine the user's speaking state and obtain the volume of the user's voice. This method is only called back when the robot is in a woken-up state.

Parameter description:

The robot's speech recognition callback implements the RecognizeListener interface, which requires the following methods to be implemented:

void onRecognizeVolume(int volume);

The volume value returns the volume of the sound recognized by the robot, and the value ranges from 0 to 30.

3.2.8 **Method signature** for determining if

the robot is speaking :

public OperationResult isSpeaking()

Method description:

This method is used to determine whether the robot is currently speaking.

Return value description:

The getResult() method of OperationResult will return the query result. If the returned value is "1", it means that the robot is talking, and "0" means otherwise.

Example code:

```
OperationResult operationReusult = speechManager.isSpeaking();
if(operationResult.getResult().equals("1")){
//TODO robot is speaking
```

```
}
```

3.2.9 Pause Speech Synthesis

Method Signature:

public OperationResult pauseSpeak()

Method description:

This method is used to pause the speech synthesis of the current robot.

3.2.10 Continue with speech synthesis (this method is not supported in overseas versions)

method signature:

public OperationResult resumeSpeak()

Method description:

This method is used to resume speech synthesis that has been paused in the current robot.

3.2.11 Stop speech synthesis (this method is not supported in overseas versions)

method signature:

public OperationResult stopSpeak()

Method description:

This method is used to suspend the current robot's speech synthesis.

3.2.12 Speech Synthesis State Callback

Method Signature:

public void setOnSpeechListener(SpeechListener speechListener)

Method description:

This callback method is triggered when the robot is performing speech synthesis.

Parameter description:

The speech synthesis state callback implements the SpeakListener interface, which requires the following implementation:

ÿÿ

void onSpeakStatus(SpeakStatus speakStatus);

The onSpeakStatus method is called back during the speech synthesis process. The progress parameter of the SpeakStatus class indicates the current synthesis progress of the sentence, with a value range of 0 <= percent <= 100.

Example code:

```
speechManager.setOnSpeechListener(new SpeakListener() {

    @Override
    public void onSpeakStatus(SpeakStatus speakStatus) {
        System.out.print("Speech synthesis in progress, progress : " +
    speakStatus.getProgress());
    }
});
```

3.2.13 Identify **the signature of the start state callback method:**

public void setOnSpeechListener(SpeechListener speechListener)

Method description:

This method will be called back when the robot begins recognition.

Parameter description:

The robot's speech recognition callback implements the RecognizeListener interface, which requires the following methods to be implemented:

```
@Override
public void onStartRecognize() {
    Log.i("Cris", "Status callback at the start of recognition");
}
```

3.2.14 Identify **the signature of the end-**

state callback method:

public void setOnSpeechListener(SpeechListener speechListener)

Method description:

This method will be called back when the robot finishes recognition.

Parameter description:

The robot's speech recognition callback implements the RecognizeListener interface, which requires the following methods to be implemented:

```
@Override
public void onStopRecognize() {
    Log.i("Cris", "Status callback when recognition
ends");
}
```

3.2.15 Identify **the signature of the error**

status callback method:

public void setOnSpeechListener(SpeechListener speechListener)

Method description:

This method will be called back when the robot makes a mistake.

Parameter description:

The robot's speech recognition callback implements the RecognizeListener interface, which requires the following methods to be implemented:

```
@Override
public void onError(int engine, int errorCode) {
    Log.i("Cris", "onError: engine ="+ engine +" errorCode ="+
```

```
errorCode);

}
```

The current recognition engine is 0 for iFlytek and 1 for DuerOS.

ErrorCode identifies the error code. For example, if engine==0, errorCode==20005 means no one is speaking. For other error codes, please refer to iFlytek's error code list.

3.2.16 Wake up and select the recognized language:

Method signature:

public OperationResult doWakeUp(WakeUpOption option)

Method description:

This method actively wakes the machine up; it can only be called within an Activity, not a Service. For overseas customers who need to select different languages for recognition during wake-up, in addition to switching the system default language, a new feature allows passing a `WakeUpOption` during wake-up. `WakeUpOption` can be configured to specify the robot's language.

Example code:

The languages currently supported by WakeUpOption are as follows:

LAG_ARABIC_INTERNATIONAL //Arabic

LAG_CHINESE_HK //Hong Kong, China

LAG_DANISH //Danish

LAG_ENGLISH_US //English

LAG_FRENCH_FRANCE //French

LAG_GERMAN //German

LAG_ITALIAN //Italian

LAG_JAPANESE //Japanese

LAG_KOREAN //Korean

LAG_POLISH //Polish

LAG_PORTUGUESE_PORTUGAL //Portuguese

LAG_SPANISH_SPAIN //Spanish

LAG_TURKISH //Turkish

```
WakeUpOption wakeUpOption = new WakeUpOption();
wakeUpOption.setLanguageType(WakeUpOption.          LAW_ARABIC_NOT
RNATIONAL   );
speechManager.doWakeUp(wakeUpOption);
```

3.3 Hardware Control

Access process:

1. Request a HardWareManager object. The request code is as follows:

```
HardWareManager hardWareManager =
(HardWareManager)getUnitManager(FuncConstant.HARDWARE_MANAGER);
```

2. Use the requested hardWareManager object to call the corresponding methods for control.

3.3.1 **Signature of the method for controlling the white light:**

public OperationResult switchWhiteLight(boolean isOpen)

Method description:

Used to turn the white light on or off.

Example code:

```
hardWareManager.switchWhiteLight(true);
```

3.3.2 Signature of LED light control **method:**

public OperationResult setLED(LED led)

Method description:

Control the robot's LED lights.

Parameter description:

The constructor for the LED class is shown below:

LED(byte part, byte mode, byte delayTime, byte randomCount)

The `part` parameter indicates the location of the LED light on the robot.

The `mode` parameter indicates the control mode of the LED light.

delayTime is only used when the LED is in blinking mode. It represents the blinking interval of the LED (note that the value of delayTime is between 1 and 255, and the unit is 100ms).

randomCount is only used when the LED is in random color blinking mode; it represents the random color to blink. Quantity (randomCount ranges from 1 to 7).

The following is an introduction to the possible values that can be passed to the part parameter:

LED.PART_ALL: All LED lights for the robot.

LED.PART_WHEEL LED lights on the robot chassis

LED.PART_LEFT_HAND: The LED light on the robot's left wing.

LED.PART_RIGHT_HAND: The LED light on the robot's right wing.

LED.PART_LEFT_HEAD: LED light on the left side of the robot's head.

LED.PART_RIGHT_HEAD: LED light on the right side of the robot's head.

The following is an introduction to the possible values that can be passed to the mode parameter:

LED.MODE_CLOSE turns off the LED lights.

LED.MODE_WHITE White LED light

LED.MODE_RED (Red LED)

LED.MODE_GREEN Green LED light

LED.MODE_PINK Pink LED light

LED.MODE_PURPLE Purple LED light

LED.MODE_BLUE Blue LED light

LED.MODE_YELLOW Yellow LED light

LED.MODE_FLICKER_WHITE is a blinking white LED (delayTime controls the blinking interval).

LED.MODE_FLICKER_RED is a flashing red LED light.

LED.MODE_FLICKER_GREEN is a flashing green LED light.

LED.MODE_FLICKER_PINK is a flashing pink LED light.

LED.MODE_FLICKER_PURPLE is a flashing purple LED.

LED.MODE_FLICKER_BLUE is a flashing blue LED light.

LED.MODE_FLICKER_YELLOW is a flashing yellow LED light.

LED.MODE_FLICKER_RANDOM causes random color blinking (randomCount can be controlled).

(Number of colors that the LED flashes randomly)

Example code:

```
hardWareManager.setLED(new LED(LED.PART_ALL,LED.
MODE_FLICKER_RANDOM,10,3));
```

3.3.3 Touch Event Callback

Method signature:

public void setOnHareWareListener(HardWareListener hareWareListener)

Method description:

When the robot's touch sensor is triggered, the touch event will trigger a callback.

Parameter description:

Touch events implement the TouchSensorListener interface, which requires the following methods to be implemented:

void onTouch(int part);

The value of part ranges from 1 to 13.

1	Right side sensor of chin	**2**	Sensor on the left side of the chin
3	Right side chest sensor	**4**	Sensor on the left side of the chest
5	Sensor on the left side of the back of the head	**6**	Sensor on the right side of the back of the head
7	Left side sensor on the back	**8**	Back right side sensor
9	Left-hand sensor	**10**	Right-hand sensor
11	Sensor at the center of the top of the head	**12**	Sensor on the right front of the head

13	Top left front sensor

Example code:

```
hardWareManager.setOnHareWareListener(new TouchSensorListener() {
    @Override
    public void onTouch(int part) {
        If (part == 11 || part == 12 || part == 13) { touchTv.setText("You
            touched the head");
        }
    }
});
```

3.3.4 Sound source localization **method signature:**

public void setOnHareWareListener(HardWareListener hareWareListener)

Method description:

When the robot is woken up by voice, a sound source localization event will be triggered.

Parameter description:

The sound source localization event implements the VoiceLocateListener interface, which requires the following methods to be implemented:

void voiceLocateResult(int angle);

angle represents the offset angle of the sound source relative to the front of the head, and is calculated in a clockwise direction.

Example code:

```
hardWareManager.setOnHareWareListener(new VoiceLocateListener() {
    @Override
    public void voiceLocateResult(int angle) {
        //TODO: Obtain the sound source localization angle and process it.
    }
});
```

3.3.5 PIR detection

method signature:

public void setOnHareWareListener(HardWareListener hareWareListener)

Method description:

The robot has a PIR module in front and behind it. When the robot's PIR module detects that someone has passed by or left in front of or behind the robot, a PIR detection event will be triggered.

Parameter description:

The PIR detection event implements the PIRListener interface, which requires the following methods to be implemented:

void onPIRCheckResult(boolean isChecked,int part);

When `isChecked` is true, it means the PIR sensor detected someone passing by; when it is false, it means the sensor detected someone leaving from in front of or behind the robot. The `part` parameter represents the location of the infrared sensor, with a value ranging from 1 to 2. 1 indicates the PIR sensor is located at the front of the robot, and 2 indicates it is located at the back of the robot.

Example code:

```
hardWareManager.setOnHareWareListener(new PIRListener() {
 @Override
 public void onPIRCheckResult(boolean isCheck,int part) {
        System.out.print((part == 1 ? "Front": "Back") + "PIR was touched"
hair"); }

});
```

3.3.6 Infrared sensor callback (infrared ranging)

method signature:

public void setOnHareWareListener(HardWareListener hareWareListener)

Method description:

The infrared sensor callback function currently returns the distance measurements from the robot's 18 infrared emitters to the obstacles. The result is shown below. A schematic diagram of the robot's infrared transmitter location is also shown:

Parameter description:

The infrared sensor-related functions implement the InfrareListener interface, which requires the following methods to be implemented:

void infrareDistance(int part,int distance);

`infrareDistance` is a method for obtaining infrared ranging results. The `part` parameter indicates the location of the infrared sensor. The values correspond one-to-one with the locations marked in the diagram above, ranging from 1 to 17, representing infrared ranging modules 1 to 17 respectively. "distance" indicates the distance from the measuring module to the obstacle, in centimeters.

Example code:

```
hardWareManager.setOnHareWareListener(new InfrareListener() {
    @Override
    public void infrareDistance(int part, int distance) {
        if (distance != 0) {
            System.out.print("The distance between part " + part + " is " + distance);
        }
    }
});
```

3.3.7 Signature of method to enable/disable

black line filtering :

public OperationResult switchBlackLineFilter(boolean isOpen)

Method description:

Wide black lines or gaps on the ground can interfere with the robot's infrared ranging function, causing the wheels to stop moving. To prevent interference and allow the robot to continue moving in such environments, the black line filtering function must be enabled. Please note: Enabling the black line filtering function will disable the robot's fall protection; avoid enabling it in environments where there is a risk of the robot falling.

3.3.8 **Method** for setting white light

brightness (signature):

public OperationResult setWhiteLightLevel(int level)

Method description:

The brightness of the robot's white light is set. The value of level ranges from 1 to 3, representing the brightness levels of the white light as energy-saving, soft, and bright, respectively.

3.3.9 Signatures of gyroscope-related function

callback **methods:**

public void setOnHareWareListener(HardWareListener hareWareListener)

Method description:

This callback interface is used to obtain the deflection angle data measured by the gyroscope device on the robot.

Parameter description:

The gyroscope-related functions implement the GyroscopeListener interface, which requires the following methods to be implemented:

void gyroscopeData(float driftAngle, float elevationAngle,
float rollAngle);

The above method is a callback of gyroscope measurement results, where driftAngle represents the yaw angle data and elevationAngle represents... Pitch angle data, rollAngle represents roll angle data.

Example code:

```
hardWareManager.setOnHareWareListener(new GyroscopeListener() {
  @Override
```

```
    public void gyroscopeData(float driftAngle, float elevationAngle, float
rollAngle) {
        //TODO: Return gyroscope measurement data}
});
```

3.3.10 Barrier Detection

Method Signature:

public void setOnHareWareListener(HardWareListener hareWareListener)

Method description:

The robot has distance detectors on its wheels, chassis, wings, and front and rear of its body. When these sensors detect an obstacle nearby, they report the obstacle status.

Parameter description:

The obstacle detection event implements the ObstacleListener interface, which requires the following methods to be implemented:

void onObstacleStatus(boolean status);

Callback parameter status: true (obstacle avoidance in progress), false (safe environment)

Example code:

```
hardWareManager.setOnHareWareListener(new ObstacleListener() {
    @Override
     public void onObstacleStatus(boolean b) {
          if (b) {
               Log.w("info", "onObstacleStatus: Obstacle avoidance in progress!");
          } else {
        Log.i("info", "onObstacleStatus: No obstacles around.");
```

```
        }

    }

});
```

3.4 Head movement control

Access process:

1. Request a HeadMotionManager object. The request code is as follows:

```
HeadMotionManager headMotionManager =
(HeadMotionManager)getUnitManager(FuncConstant.HEADMOTION_MANAGER); 2. Use the requested headMotionManager object to
```

call the corresponding methods for control.

3.4.1 Signature of relative

angle motion **method:**

public OperationResult

doRelativeAngleMotion(RelativeAngleHeadMotion relativeAngleHeadMotion)

Method description:

Control the robot's head to rotate a specified angle in a designated direction relative to its current position.

Parameter description:

The `**RelativeAngleHeadMotion**` class has two parameters: `action` and `angle`. `action` represents the motion...

Robot head movement pattern, where angle represents the relative angle of the robot's head movement.

The following is an introduction to the possible values that can be passed to the action parameter:

RelativeAngleHeadMotion.ACTION_STOP stops the motion.

RelativeAngleHeadMotion.ACTION_UP upward movement

RelativeAngleHeadMotion.ACTION_DOWN moves downwards.

RelativeAngleHeadMotion.ACTION_LEFT moves to the left.

RelativeAngleHeadMotion.ACTION_RIGHT moves to the right.

RelativeAngleHeadMotion.ACTION_LEFTUP moves to the upper left.

RelativeAngleHeadMotion.ACTION_RIGHTUP moves to the upper right.

RelativeAngleHeadMotion.ACTION_LEFTDOWN moves to the lower left.

RelativeAngleHeadMotion.ACTION_RIGHTDOWN moves to the lower right.

Example code:

```
RelativeAngleHeadMotion relativeAngleHeadMotion = new
RelativeAngleHeadMotion(
        RelativeAngleHeadMotion.ACTION_RIGHT,30
);
headMotionManager.doRelativeAngleMotion(relativeAngleHeadMotion);
```

3.4.2 Signature for Absolute Angular Motion **Method:**

public OperationResult

doAbsoluteAngleMotion(AbsoluteAngleHeadMotion absoluteAngleHeadMotion)

Method description:

Control the robot's head to rotate to a specified angle. The absolute angle in the horizontal direction ranges from 0 to 180 degrees, increasing from left to right. The absolute angle in the vertical direction ranges from 7 to 30 degrees, increasing from lower to higher values.

Parameter description:

The AbsoluteAngleHeadMotion class has two parameters: action and angle. Action represents the robot's head motion pattern, and angle represents the absolute angle of the robot's head motion.

The following is an introduction to the possible values that can be passed to the action parameter:

AbsoluteAngleHeadMotion.ACTION_VERTICAL Vertical movement

AbsoluteAngleHeadMotion.ACTION_HORIZONTAL Horizontal movement

Example code:

```
AbsoluteAngleHeadMotion absoluteAngleHeadMotion = new
AbsoluteAngleHeadMotion(
        AbsoluteAngleHeadMotion.ACTION_HORIZONTAL,130
```

```
);
headMotionManager.doAbsoluteAngleMotion(absoluteAngleHeadMotion);
```

3.4.3 **Signature of the horizontal**

centering locking method:

public OperationResult dohorizontalCenterLockMotion()

Method description:

Control the robot's head to rotate to a horizontal 90-degree position and lock the robot's horizontal motor. (Once locked, the robot's head cannot be manually rotated horizontally).

Example code:

```
headMotionManager.dohorizontalCenterLockMotion();
```

3.4.4 Absolute Angle Positioning Operation

Method Signature:

public OperationResult

doAbsoluteLocateMotion(LocateAbsoluteAngleHeadMotion locateAbsoluteAngleHeadMotion)

Method description:

The robot's head is controlled to rotate to a specified angle, and after the action is completed, it is determined whether to lock the head motors. The absolute angle in the horizontal direction ranges from 0 to 180 degrees, increasing from left to right. The absolute angle in the vertical direction ranges from 7 to 30 degrees, increasing from lower to higher values. The difference between this method and the head absolute angle movement method is that this method can determine whether to lock the motors. This method does not require specifying the rotation mode; instead, it directly specifies the absolute angle values in the horizontal and vertical directions.

Parameter description:

The **LocateAbsoluteAngleHeadMotion** class has three parameters: action, horizontalAngle, and horizontalAngle.

And verticalAngle. action indicates the robot's head locking mode, horizontalAngle indicates the absolute angle of the robot's head movement in the horizontal direction, and verticalAngle indicates the absolute angle of the robot's head movement in the vertical direction.

The following is an introduction to the possible values that can be passed to the action parameter:

LocateAbsoluteAngleHeadMotion.ACTION_NO_LOCK Motor not locked.

LocateAbsoluteAngleHeadMotion.ACTION_HORIZONTAL_LOCK (Motor horizontal direction lock)

LocateAbsoluteAngleHeadMotion.ACTION_VERTICAL_LOCK locks the motor in the vertical direction.

LocateAbsoluteAngleHeadMotion.ACTION_BOTH_LOCK locks the motor in both the horizontal and vertical directions.

Example code:

```
LocateAbsoluteAngleHeadMotion locateAbsoluteAngleHeadMotion = new
LocateAbsoluteAngleHeadMotion(
        LocateAbsoluteAngleHeadMotion.ACTION_BOTH_LOCK,30,15
);
headMotionManager.doAbsoluteLocateMotion(locateAbsoluteAngleHeadM
otion);
```

3.4.5 Relative Angle Positioning Operation

Method Signature:

public OperationResult

doRelativeLocateMotion(LocateRelativeAngleHeadMotion locateRelativeAngleHeadMotion)

Method description:

Control the robot's head to rotate a certain angle relative to its current position, and decide whether to power the head after the rotation is complete.
The machine is locked.

Parameter description:

The **LocateRelativeAngleHeadMotion** class has five parameters: action, horizontalAngle, and horizontalAngle.
The parameters are: verticalAngle, horizontalDirection, and verticalDirection. `action` indicates the robot's head lock mode;
`horizontalAngle` represents the horizontal relative angle of the robot's head movement; `verticalAngle` represents the vertical relative
angle of the robot's head movement; `horizontalDirection` represents the horizontal direction of the robot's head
movement; and `verticalDirection` represents the vertical direction of the robot's head movement.

The following is an introduction to the possible values that can be passed to the action parameter:

LocateRelativeAngleHeadMotion.ACTION_NO_LOCK Motor not locked

LocateRelativeAngleHeadMotion.ACTION_HORIZONTAL_LOCK (Motor horizontal direction lock)

LocateRelativeAngleHeadMotion.ACTION_VERTICAL_LOCK locks the motor in the vertical direction.

LocateRelativeAngleHeadMotion.ACTION_BOTH_LOCK locks the motor in both the horizontal and vertical directions.

The following is an introduction to the possible values that can be passed to the horizontalDirection parameter:

LocateRelativeAngleHeadMotion.DIRECTION_LEFT moves horizontally to the left.

LocateRelativeAngleHeadMotion.DIRECTION_RIGHT (Horizontal movement to the right)

LocateRelativeAngleHeadMotion.DIRECTION_UP Vertical upward movement

LocateRelativeAngleHeadMotion.DIRECTION_DOWN Vertical downward movement

Example code:

```
LocateRelativeAngleHeadMotion locateRelativeAngleHeadMotion = new
LocateRelativeAngleHeadMotion(
        LocateRelativeAngleHeadMotion.ACTION_BOTH_LOCK,(byte) 30,
(byte) 15,
        LocateRelativeAngleHeadMotion.DIRECTION_LEFT
        ,LocateRelativeAngleHeadMotion.DIRECTION_UP
);
headMotionManager.doRelativeLocateMotion(locateRelativeAngleHeadM
otion);
```

3.5 Wing Motion Control

Access process:

1. Request a WingMotionManager object. The request code is as follows:

```
WingMotionManager wingMotionManager=
(WingMotionManager)getUnitManager(FuncConstant.WINGMOTION_MANAGER );
```

2. Use the requested wingMotionManager object to call the corresponding methods for control.

3.5.1 Signature for the angleless motion **method:**

public OperationResult doNoAngleMotion(NoAngleWingMotion noAngleWingMotion)

Method description:

Control the robot's wings to make non-angular movements, such as raising the arm and returning the wings to their original position.

Parameter description:

The NoAngleWingMotion class has three parameters: action, part, and speed. action represents the rotation mode of the robot's wing, part represents the position of the robot's wing (left or right wing), and speed represents the speed of the robot's wing movement, with a value ranging from 1 to 10.

The following is an introduction to the possible values that can be passed to the action parameter:

NoAngleWingMotion.ACTION_UP (Wings rotate upwards)

NoAngleWingMotion.ACTION_DOWN (Wings rotate downwards)

NoAngleWingMotion.ACTION_STOP stops the wings from rotating.

NoAngleWingMotion.ACTION_RESET returns the wings to their default position (i.e., wings pointing vertically downwards).

The following is an introduction to the possible values that can be passed to the part parameter:

NoAngleWingMotion.PART_LEFT Left Wing Operation

NoAngleWingMotion.PART_RIGHT Right Wing Operation

NoAngleWingMotion.PART_BOTH: Operate both wings simultaneously.

Example code:

```
NoAngleWingMotion noAngleWingMotion = new
NoAngleWingMotion(NoAngleWingMotion.PART_BOTH,
                  5,NoAngleWingMotion.ACTION_UP);
wingMotionManager.doNoAngleMotion(noAngleWingMotion);
```

3.5.2 Signature for Relative Angular Motion **Method:**

public OperationResult

doRelativeAngleMotion(RelativeAngleWingMotion relativeAngleWingMotion)

Method description:

Control the robot's wings to rotate a specified angle in a designated direction relative to its current position.

Parameter description:

The RelativeAngleWingMotion class has four parameters: action, part, speed, and angle. action represents the rotation mode of the robot's wing, part represents the position of the robot's wing (left or right wing), speed represents the speed of the robot's wing movement, with a value range of 1 to 8, and angle represents the relative angle of the robot's wing rotation, with a value range of 0 to 270, and the angle gradually increases in the counterclockwise direction.

The following is an introduction to the possible values that can be passed to the action parameter:

RelativeAngleWingMotion.ACTION_UP: Wings rotate upwards.

RelativeAngleWingMotion.ACTION_DOWN (Wings rotate downwards)

The following is an introduction to the possible values that can be passed to the part parameter:

RelativeAngleWingMotion.PART_LEFT Left Wing Operation

RelativeAngleWingMotion.PART_RIGHT Right Wing Operation

RelativeAngleWingMotion.PART_BOTH: Operate both wings simultaneously.

Example code:

```
RelativeAngleWingMotion relativeAngleWingMotion = new
RelativeAngleWingMotion( RelativeAngleWingMotion.PART_LEFT,5,
RelativeAngleWingMotion.ACTION_UP,
                30
);
wingMotionManager.doRelativeAngleMotion(relativeAngleWingMotion);
```

3.5.3 Absolute Angular Motion

Method Signature:

public OperationResult

doAbsoluteAngleMotion(AbsoluteAngleWingMotion absoluteAngleWingMotion)

Method description:

Control the robot's wings to rotate to a specified angle and position.

Parameter description:

The **`AbsoluteAngleWingMotion`** class has three parameters: `speed`, `part`, and `angle`. `speed` represents...

This indicates the speed of the robot's wing movement, with a value ranging from 1 to 8. `part` indicates the position of the robot's wing (left or right wing).

(wing), angle represents the absolute angle of rotation of the robot's wings, with a value range of 0~270, and the angle gradually increases in the counterclockwise direction.

The following is an introduction to the possible values that can be passed to the part parameter:

AbsoluteAngleWingMotion.PART_LEFT Left Wing Operation

AbsoluteAngleWingMotion.PART_RIGHT Right Wing Operation

AbsoluteAngleWingMotion.PART_BOTH: Operate both wings simultaneously.

Example code:

```
AbsoluteAngleWingMotion absoluteAngleWingMotion = new
AbsoluteAngleWingMotion(
        AbsoluteAngleWingMotion.PART_LEFT,5,0
);
wingMotionManager.doAbsoluteAngleMotion(absoluteAngleWingMotion);
```

3.6 Wheel Motion Control

Access process:

1. Request a WheelMotionManager object. The request code is as follows:

```
WheelMotionManager wheelMotionManager =
(WheelMotionManager)getUnitManager(FuncConstant.WHEELMOTION_MANAG ER);
```

2. Use the requested wheelMotionManager object to call the corresponding methods for control.

3.6.1 Signature for the angleless motion **method:**

public OperationResult doNoAngleMotion(NoAngleWheelMotion noAngleWheelMotion)

Method description:

Control the robot's wheels to make movements without angles, such as moving forward, backward, or to the left.

Parameter description:

The **`NoAngleWheelMotion`** class has three parameters: `speed`, `action`, and `duration`. `speed` represents the robot wheel's movement speed, ranging from 1 to 10; `action` represents the robot wheel's movement mode; and `duration` represents...

The robot's wheels move for a specified time, in units of 100ms. When duration is 0, it means the robot will remain in motion until a stop command is received.

The following is an introduction to the possible values that can be passed to the action parameter:

NoAngleWheelMotion.ACTION_FORWARD Forward

NoAngleWheelMotion.ACTION_LEFT spins in place to the left.

NoAngleWheelMotion.ACTION_RIGHT Spin in place to the right

NoAngleWheelMotion.ACTION_TURN_LEFT Left Turn

NoAngleWheelMotion.ACTION_TURN_RIGHT (Turn right)

NoAngleWheelMotion.ACTION_STOP_TURN Stop turning

NoAngleWheelMotion.ACTION_STOP_RUN (Stop walking)

Example code:

```
NoAngleWheelMotion noAngleWheelMotion2 = new NoAngleWheelMotion(
        NoAngleWheelMotion.ACTION_FORWARD_RUN, 5,3000
);
wheelMotionManager.doNoAngleMotion(noAngleWheelMotion2);
```

3.6.2 Signature for Relative Angular Motion **Method:**

public OperationResult

doRelativeAngleMotion(RelativeAngleWheelMotion relativeAngleWheelMotion)

Method description:

Control the robot's wheels to rotate a specified angle in a designated direction relative to its current position.

Parameter description:

The **`RelativeAngleWheelMotion`** class has three parameters: `speed`, `action`, and `angle`. `speed` represents the speed of the robot's wheels, with a value ranging from 1 to 10. `action` represents the movement pattern of the robot's wheels, and `angle` represents the relative angle of rotation of the robot's wheels.

The following is an introduction to the possible values that can be passed to the action parameter:

RelativeAngleWheelMotion.TURN_LEFT turns left

RelativeAngleWheelMotion.TURN_RIGHT (Turn right)

RelativeAngleWheelMotion.TURN_STOP stops the rotation.

Example code:

```
RelativeAngleWheelMotion relativeAngleWheelMotion = new
RelativeAngleWheelMotion(
        RelativeAngleWheelMotion.TURN_LEFT, 5,90
);
wheelMotionManager.doRelativeAngleMotion(relativeAngleWheelMotion
);
```

3.6.3 Distance Motion

Method Signature:

public OperationResult doDistanceMotion(DistanceWheelMotion distanceWheelMotion)

Method description:

Control the robot to move a specified distance in a specified direction.

Parameter description:

The DistanceWheelMotion class has three parameters: speed, action, and distance. Speed represents the speed of the robot's wheels, with a value ranging from 1 to 10. Action represents the movement mode of the robot's wheels, and distance represents the distance the robot moves, in centimeters.

The following is an introduction to the possible values that can be passed to the action parameter:

DistanceWheelMotion.ACTION_FORWARD_RUN

DistanceWheelMotion.ACTION_STOP_RUN stops walking.

Example code:

```
DistanceWheelMotion distanceWheelMotion = new
DistanceWheelMotion(
        DistanceWheelMotion.ACTION_FORWARD_RUN, 5,100
);
wheelMotionManager.doDistanceMotion(distanceWheelMotion);
```

3.6.4 Description of Wheel Movement State

Callback **Method:**

After the robot's wheels move, it will revert to the wheel's movement state.

Parameter description:

The callback state s indicates that s="0" represents the motion has stopped, while non-"0" indicates the motion state.

Example code:

```
wheelMotionManager.setWheelMotionListener(new
WheelMotionManager.WheelMotionListener() {
        @Override
        public void onWheelStatus(String s) {
                Log.i("Cris", "onWheelStatus: s="+s);
        }
});
```

3.7 System Management

Access Requirements: Some features can only be accessed by apps that have been signed by the system. The features that require system signature to access will be marked below.

Access process:

1. Request a SystemManager object. The request code is as follows:

```
SystemManager systemManager=
 (SystemManager)getUnitManager(FuncConstant. SYSTEM_MANAGER);
```

2. Use the requested SystemManager object to call the corresponding methods for control.

3.7.1 **Method signature** for obtaining device ID :

```
public String getDeviceId();
```

Method description:

Get the device ID number of the current robot.

Example code:

```
systemManager.getDeviceId();
```

3.7.2 **Method signature for returning to the**

screensaver animation interface:

public OperationResult doHomeAction()

Method description:

Make the robot's display jump to the desktop screensaver animation interface.

Return value description:

The ErrorCode of OperationResult may return the ErrorCode.FAIL_APP_HAS_LOCKED error.

This indicates that an app is currently locked and cannot be returned to the screensaver animation screen.

Example code:

```
systemManager.doHomeAction();
```

3.7.3 Facial Expression

Control **Method Signature:**

public OperationResult showEmotion(EmotionsType emotion)

Method description:

This function enables the robot to display a specified expression. It will only take effect when the currently displayed page is a human face interface.

Parameter description:

The following is an introduction to the possible values that can be passed to the `emotion` parameter:

EmotionsType.ARROGANCE Arrogance

EmotionsType.SURPRISE Surprise

EmotionsType.WHISTLE (Whistle)

EmotionsType.LAUGHTER (laughing)

EmotionsType.GOODBYE Farewell

EmotionsType.SHY (shy)

EmotionsType.SWEAT Sweat

EmotionsType.SNICKER (Evil Smile)

EmotionsType.PICKNOSE ÿÿ

EmotionsType.CRY (crying)

EmotionsType.ABUSE (Insults/Swearing)

EmotionsType.ANGRY Angry

EmotionsType.KISS (Kiss)

EmotionsType.SLEEP (Sleeping)

EmotionsType.SMILE (Smile)

EmotionsType.GRIEVANCE (Feeling wronged)

EmotionsType.QUESTION Question

EmotionsType.FAINT ÿ

EmotionsType.PRISE Like

EmotionsType.NORMAL (default)

Example code:

```
systemManager.showEmotion(EmotionsType. PRISE);
```

3.7.4 Signature of the method for

obtaining battery level :

public int getBatteryValue()

Method description:

Used to obtain the current battery level of the robot.

Return value description:

The `result` field of `OperationResult` returns the battery level, which is an integer.

3.7.5 **Method signature** for

obtaining battery status :

public int getBatteryStatus()

Method description:

Used to obtain the current battery charging status of the robot.

Return value description:

The `result` field of `OperationResult` returns the robot's charging status. There are three possible results: **not charging.**

SystemManager.STATUS_NORMAL

SystemManager.STATUS_CHARGE_PILE is charging. **Using charging stations for robots**

SystemManager.SATUS_CHARGE_LINE ÿ **Charge the robot using the power cord.**

3.7.6 **Method signature** for monitoring

security home alarms :

public void setOnIDarlingListener(IDarlingListener iDarlingListener)

Method description:

If Security Home has its arming policy enabled, this callback method will be invoked when a Security Home alarm event is triggered.

Parameter description:

The home security alert callback implements the ICharlingListener interface, which requires the following methods to be implemented:

void onAlarm(int type);

The type indicates the home security alarm mode. When the type value is 1, it means that an obstruction alarm has been triggered. When it is 2, it means that an intrusion alarm has been triggered. When it is 3, it means that an out-of-bounds alarm has been triggered.

3.7.7 Method signature for obtaining the main

controller version number :

public String getMainServiceVersion()

Method description:

Get the current system's main control program version number.

3.7.8 **Method signature** for showing/hiding

system floating buttons :

public OperationResult switchFloatBar(boolean isShow,String className)

Method description:

Show/hide the current system's floating button. This method can only be called within an Activity; calling it from a Service will not work.

Parameter description:

Does **isShow** display the system's floating button?

`className` is the fully qualified class name of the current Activity, which can be obtained using the `getClass().getName()` method.

3.7.9 Show/hide system floating button (to avoid it flashing briefly when switching between activities within the app)

method signature:

public OperationResult switchFloatBarInApplication(boolean isShow)

Method description:

Show/hide the current system's floating buttons.

Parameter description:

Does **isShow** display the system's floating button?

3.7.10 **Method signature** for querying robot type :

public int getRobotType();

Method description:

Get robot type

Method return description:

SystemManager.ROBOT_TYPE_B; (Type B, Little Sprite)

SystemManager.ROBOT_TYPE_D; (Type D, King Kong)

SystemManager.ROBOT_TYPE_DESKTOP; (Desktop robot, mini sprite)

3.8 Multimedia Management

Access process:

1. Request an HDCameraManager object; the request code is as follows:

```
HDCameraManager hdCameraManager =
(HDCameraManager)getUnitManager(FuncConstant.HDCAMERA_MANAGER);
```

2. Use the requested hdCameraManager object to call the corresponding methods for control.

3.8.1 Obtaining the signatures of audio and video

stream callback **methods:**

public void setMediaListener(MediaListener mediaListener)

Method description:

The robot acquires audio and video stream data using its high-definition camera and microphone.

Precautions:

1. High-definition camera microphones do not support echo cancellation, so there will be very sharp noise when recording locally and playing it back at the same time. You need to be careful when using audio.

Parameter description:

The MediaStreamListener interface is used to retrieve audio and video stream callbacks. This interface requires the following methods to be implemented:

```
void getVideoStream(byte[] data); void
getAudioStream(byte[] data);
```

The two methods above correspond to video stream data callback and audio stream data callback methods, respectively. For details on processing the obtained audio and video streams, please refer to the code in the provided demo.

3.8.2 **Method signature for**

opening a media stream:

public OperationResult openStream(StreamOption streamOption)

Method description:

The method to open a media stream can only retrieve the callback data of the audio and video streams after the method is called.

Parameter description:

StreamOption has the following parameters:

The `type` parameter indicates the decoding method used for the video, and its possible values are:

StreamOption.TYPE_HARDWARE_DECORD Hardware decoding (default value)

StreamOption.TYPE_SOFTWARE_DECORD software decoding

isJustIframe indicates whether to return only I-frame data (note that it is the English letter I, not the number 1), and the default value is false.

The `channel` parameter indicates the bitstream format of the returned video, and its possible values are:

StreamOption.MAIN_STREAM: The main bitstream, returning a video resolution of 1280*720 (default).

The StreamOption.SUB_STREAM sub-stream returns a video resolution of 640*480.

Example code is as follows:

```
StreamOption streamOption = new StreamOption();
                                                    MAIN_STREAM    );
                                                       HARDWARE_DECODE    );




                                valueOf


streamOption.setChannel(StreamOption.streamOption.setDecodType(StreamOption.streamOption.setJustIframe(fa
```

Currently, a maximum of two video streams can be pulled at once. The `hdCameraManager.openStream(streamOption)` method returns an `OperationResult` object. The `getResult()` method of `OperationResult` retrieves the pointer to the opened stream. If the stream was opened successfully, it returns a string greater than 0, which needs to be converted to an `int` type. This pointer is used when closing the stream and is passed in.

3.8.3 **Signature of the method to**

close the media stream:

public OperationResult closeStream(int handle)

Method description:

Methods to close the media stream. If you have called the `openStream` method, please remember to close it when the interface is destroyed. This method must be called to close the media stream; otherwise, it may cause unpredictable risks such as memory leaks.

Example code

`hdCameraManager.closeStream(handle)`, where `handle` represents the pointer returned when opening the stream.

3.8.4 Obtain **the signature of the face**

recognition callback method:

public void setMediaListener(MediaListener mediaListener)

Method description:

This callback method is triggered when the robot recognizes a face in front of it. If the current user has already registered information in the family member app, the robot will return the user's registered information along with the user's information if it recognizes the user. Please note that registering a user in the family member app does not guarantee that the robot will recognize that user. The accuracy of recognition is also affected by factors such as ambient lighting and the angle of the face. This method will not be called back in an offline environment.

Parameter description:

The FaceRecognizeListener interface is used to retrieve face recognition callbacks. This interface requires the following methods to be implemented:

void recognizeResult(List<FaceRecognizeBean> faceRecognizeBean);

FaceRecognizeBean contains the location of the recognized face on the screen and the user's information on family members.

The relevant personal information (if any) includes the following parameters:

private int w (width of the face recognition image, currently defaults to 1280) private String birthday (user's birthday information, which must be entered into the family member database) private double bottom (percentage of the screen occupied by the bottom of the face) private String gender (user's gender information, which must be entered into the family member database) private double left (percentage of the screen occupied by the leftmost position of the face)

private String qlink (reserved field) private double right (percentage of the screen occupied by the rightmost position of the face) private String user (user's name information, which must be entered into the family member database) private int h (height of the face recognition image, currently defaults to 720) private double top (percentage of the screen occupied by the top of the face)

Example code:

```
hdCameraManager.setMediaListener(new FaceRecognizeListener() {
    @Override
    public void recognizeResult(List<FaceRecognizeBean>
faceRecognizeBean) {
        Log.i("info","Face recognition data obtained");
    }
});
```

3.8.5 **Method signature** for capturing

images from a video stream :

public Bitmap getVideoImage()

Method description:

This method can be called if you need to capture images from the robot's video stream. This method is used in conjunction with method 3.9.4.

When used together, it can be used to capture human faces.

Example code:

```
hdCameraManager.setMediaListener(new FaceRecognizeListener() {
 @Override
 public void recognizeResult(FaceRecognizeBean
faceRecognizeBean) {
        Bitmap bitmap = mediaManager.getVideoImage();
          if(bitmap != null){
              Log.i("info","Received face recognition image");
            }
 }
});
```

3.9 Projector Control

Access process:

1. Request a ProjectorManager object. The request code is as follows:

```
PojectorManager projectorManager =
(ProjectorManager)getUnitManager(FuncConstant.PROJECTOR_MANAGER);
```

2. Use the requested projectorManager object to call the corresponding methods for control.

3.9.1 Projector Switching

Method **Signature:**

public OperationResult switchProjector(boolean isOpen)

Method description:

This is used to control the projector to turn on and off. Please note that each time you turn the projector on and off, there must be an interval of at least 12 seconds, otherwise the command will not be executed.

Parameter description:

**`isOpen** true` means the projector is on; otherwise, it is off.

Example code:

```
projectorMananger.switchProjector(true);
```

3.9.2 Set the projector mirroring

method signature:

public OperationResult setMirror(int value)

Method description:

Used to configure the projector mirroring.

Parameter description:

The value ranges from 0 to 3, and the following mirroring modes are available:

ProjectorManager.MIRROR_CLOSE 0: No flipping, default mode.

ProjectorManager.MIRROR_LR 1: Flip horizontally

ProjectorManager.MIRROR_UD 2 (Flip vertically)

ProjectorManager.MIRROR_ALL 3 flips the projector up, down, left, and right.

Example code:

```
projectorMananger.setMirror(ProjectorManager.MIRROR_ALL);
```

3.9.3 **Method** for setting projector trapezoidal horizontal correction value (signature):

public OperationResult setTrapezoidH(int value)

Method description:

Used to set the trapezoidal horizontal correction value for the projector.

Parameter description:

The valid value range for **Value** is -30 to 30.

Example code:

```
projectorMananger.setTrapezoidH(10);
```

3.9.4 **Method** for setting projector trapezoidal vertical correction value (signature):

public OperationResult setTrapezoidV(int value)

Method description:

Used to set the trapezoidal vertical correction value for the projector.

Parameter description:

The valid value range for **Value** is -20 to 30.

Example code:

```
projectorMananger.setTrapezoidV(10);
```

3.9.5 **Method** for setting projector contrast (signature):

public OperationResult setContrast(int value)

Method description:

Used to adjust the contrast of the projector image.

Parameter description:

The valid value range for **Value** is -15 to 15.

Example code:

```
projectorMananger.setContrast(10);
```

3.9.6 **Method** for setting projector brightness (signature):

public OperationResult setBright(int value)

Method description:

Used to adjust the brightness of the projector screen.

Parameter description:

The valid value range for **Value** is -31 to 31.

Example code:

```
projectorMananger.setBright(10);
```

3.9.7 **Method signature** for setting

projector color values :

public OperationResult setColor(int value)

Method description:

Used to set the color values of the projector screen.

Parameter description:

The valid value range for **Value** is -15 to 15.

Example code:

```
projectorMananger.setColor(10);
```

3.9.8 Method for setting projector image

saturation **signature:**

public OperationResult setSaturation(int value)

Method description:

Used to set the saturation of the projector image.

Parameter description:

The valid value range for **Value** is -15 to 15.

Example code:

```
projectorMananger.setSaturation(10);
```

3.9.9 **Method** for setting projector image

sharpness (signature):

public OperationResult setAcuity(int value)

Method description:

Used to adjust the sharpness of the projector image.

Parameter description:

The valid value range for **Value** is 0~6.

Example code:

```
projectorMananger.setAcuity(4);
```

3.9.10 Projector Expert Mode Optical Axis Adjustment

Method (Signature):

public OperationResult setExpertAxis(int value)

Method description:

Used to set the optical axis in the projector's expert mode.

Parameter description:

Value 1 Enable Settings 2 Increase Value 3 Decrease Value 4 Exit Settings Without Saving 5 Save and Exit Settings

Example code:

```
projectorMananger.setExpertAxis(1);
```

3.9.11 Projector Expert Mode Phase Adjustment

Method Signature:

public OperationResult setExpertPhase(int value)

Method description:

Used to set the phase in the projector's expert mode.

Parameter description:

Value 1 Enable Settings 2 Increase Value 3 Decrease Value 4 Exit Settings Without Saving 5 Save and Exit Settings

Example code:

```
projectorMananger.setExpertPhase(1);
```

3.9.12 **Method signature** for setting

projector mode :

public OperationResult setMode(int mode)

Method description:

Used to set the projection mode of the projector.

Parameter description:

Value ProjectorManager.MODE_WALL 1 Wall projection mode

ProjectorManager.MODE_CEILING 2 Ceiling projection mode

Example code:

```
projectorMananger.setMode(ProjectorManager.MODE_CEILING);
```

3.9.13 Query projector data and status

Method signature:

public OperationResult queryConfig(String configName)

Method description:

Used to check whether the projector is currently on or off, and related settings such as sharpness, contrast, and color value.
The values set, etc.

configName may have the following values:

Query horizontal trapezoidal values using **Query switch status**
ProjectorManager.CONFIG_SWITCH and ProjectorManager.CONFIG_TRAPEZOIDH.
ProjectorManager.CONFIG_TRAPEZOIDV Query vertical trapezoidal values
ProjectorManager.CONFIG_CONTRAST query contrast value
Query saturation values using **Query brightness value**
`ProjectorManager.CONFIG_BRIGHT`, **Query chromaticity value**
`ProjectorManager.CONFIG_COLOR`, and `ProjectorManager.CONFIG_SATURATION`.
ProjectorManager.CONFIG_ACUITY **Query sharpness value**
ProjectorManager.CONFIG_MODE **Query mode**
ProjectorManager.CONFIG_MIRROR **Query mirror**

Example code:

```
OperationResult operationResult =
projectorMananger.queryConfig(ProjectorManager.CONFIG_SWITCH);
if(operationResult.getResult().equals("1")){
```

```
        Log.e(“info”,”Projector is on”)
}
```

3.9.14 Projector Interface Callback

Method Signature:

public void setOnProjectorListener(OnProjectorListener projectorListener)

Method description:

The monitoring interface related to the projector is used to monitor for errors that occur when adjusting the projector in expert mode.

Example code:

```
projectorManager.setOnProjectorListener(new
        ProjectorManager.ProjectorListener() {
            @Override
            public void expertModeError(ProjectorExpertMode
projectorExpertMode){
            //TODO Projector expert mode adjustment error callback
            }
});
```

3.10 Modular Motion Function Control

Access process:

1. Request a ModularMotionManager object. The request code is as follows:

```
ModularMotionManager modularMotionManager=
 (ModularMotionManager)getUnitManager(FuncConstant.
```

2. Use the requested modularMotionManager object to call the corresponding methods for control.

3.10.1 **Signature for** enabling/disabling free movement :

public OperationResult switchWander(boolean isOpen)

Method description:

Control the robot's free-walking function to turn it on and off.

Return value description:

The ErrorCode of OperationResult may have the following return values:

ErrorCode.FAIL_MOTION_LOCKED: The robot is currently performing other motion-related functions, causing motion control to be locked and unable to respond to commands.

ErrorCode.FAIL_NO_PERMISSION: The current user group does not have permission to perform this function.

ErrorCode.FAIL_IS_CHARGE: The robot is currently charging and cannot access the corresponding functions.

3.10.2 **Signature for enabling/disabling automatic charging:**

public OperationResult switchCharge(boolean isOpen)

Method description:

Control the robot's automatic charging function to turn it on and off.

Return value description:

The ErrorCode of OperationResult may have the following return values:

ErrorCode.FAIL_MOTION_LOCKED: The robot is currently performing other motion-related functions, causing motion control to be locked and unable to respond to commands.

ErrorCode.FAIL_NO_PERMISSION: The current user group does not have permission to perform this function.

ErrorCode.FAIL_IS_CHARGE: The robot is currently charging and cannot access the corresponding functions.

ErrorCode.FAIL_NO_CHARGE_PILE: The robot could not find a charging station. Please check if the charging station is plugged in or has been added to the smart network.

3.10.3 Method signature for obtaining the free-walking switch status :

public OperationResult getWanderStatus()

Method description:

Determine whether the robot's free-walking function is enabled.

Return value description:

The `getResult()` method of `OperationResult` will return the query result. If the returned value is "1", the table...

"0" indicates the roaming switch is on, and "0" indicates it is off.

Example code:

```
OperationResult operationReusult =
modularMotionManager.getWanderStatus();
if(operationResult.getResult().equals("1")){
//TODO: The wandering function is enabled.
}
```

3.10.4 Method signature for obtaining automatic charging switch status :

public OperationResult getAutoChargeStatus()

Method description:

Determine if the robot's automatic charging function is enabled.

Return value description:

The `getResult()` method of `OperationResult` will return the query result. If the returned value is "1", the table...

"0" indicates that the automatic charging switch is on, and "0" indicates that it is off.

Example code:

```
OperationResult operationReusult = modularMotionManager.
getAutoChargeStatus();
if(operationResult.getResult().equals("1")){
//TODO automatic charging function is turned on
}
}
```

3.11 Preprocessing Instructions

Preprocessing directives are instructions that execute immediately upon the establishment of a connection between an Activity or Service component and the main controller. Preprocessing directives are configured in the AndroidManifest.xml file using the `<meta-data>` tag, specifically under the `<application>`, `<activity>`, and `<service>` tags, each with a different scope. Below is a diagram illustrating preprocessing directives in the AndroidManifest.xml file:

```
<application android:allowBackup="true" android:icon="@mipmap/ic_launcher"
    android:label="QihanOpenSDK Demo" android:supportsRtl="true" android:theme=
    >
    <meta-data android:name="SPEECH_MODE" android:value="1"/>
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
        <meta-data android:name="FORBID_PIR" android:value="true"/>
    </activity>
    <activity android:name=".VideoActivity"
        android:hardwareAccelerated="true">
        <meta-data android:name="config_record" android:value="false"/>
    </activity>

    <activity android:name=".FaceRecognizeActivity"/>
    <service android:name=".MainService">
        <meta-data android:name="RECOGNIZE_MODE" android:value="1"/>
        </service>
</application>
```

The code snippet shown by the red arrow is a preprocessing directive.

Preprocessor directives configured under the `<application>` tag have a scope of all Activities and Services in the current application. It is a global attribute, which is equivalent to configuring the same preprocessor directive for each Activity or Service. Preprocessor directives configured under the `<activity>` tag take effect when the current Activity is visible, and are invalidated when the onStop method of the current Activity is called. Preprocessor directives configured under the `<service>` tag take effect when the Service is created, and are invalidated when the onDestroy method of the Service is called.

Preprocessor directives under `<application>` will be overridden by the same preprocessor directives under `<activity>` or `<service>`. If you need to use preprocessor directives under `<application>` or `<service>`, please be very careful. Because services can run in the background, once configured, preprocessor directives will remain in effect as long as the service is alive, affecting the interaction logic of the chatbot itself and the interactions of other third-party applications. Therefore, avoid using preprocessor directives under `<application>` or `<service>` unless absolutely necessary.

Note: Due to the limitations of the preprocessing instruction's working mechanism, reinstalling or uninstalling the app under the development system may cause the robot to malfunction due to the influence of the preprocessing instructions. This phenomenon is only related to the app's reinstallation and uninstallation and will not affect the normal use of the app after it is launched.

3.11.1 Recording switch

method signature:

<meta-data android:name="CONFIG_RECORD" android:value="true"/>

Method description:

This command is required if the robot needs to use the recording function. If this command is successful, the robot's ears will light up blue. Please be sure to configure it according to the above instructions; otherwise, the main control unit will restart, and the connection between the app and the main control unit will be lost.

3.11.2 Pattern Recognition

Method Signature:

<meta-data android:name="RECOGNIZE_MODE" android:value="1"/>

Method description:

The robot's voice recognition mode can currently only be set to truncated mode. Please refer to section 3.2.6 for details. bright.

3.11.3 Voice Mode

Method Signature:

<meta-data android:name="SPEECH_MODE" android:value="1"/>

Method description:

The robot's voice recognition has two built-in modes: 1. After the robot is woken up, if no one is detected speaking for more than 10 seconds, it will automatically enter a sleep state. This is the robot's default mode. 2. After the robot is woken up, if no one is detected speaking for a short period of time (about 2 to 3 seconds), it will immediately enter a sleep state. When the robot recognizes the user's words, it will also enter a sleep state. In this mode, the user needs to wake up the robot again after each conversation to continue communicating with it.

When android:value="1", the robot will enter mode two as described above.

3.11.4 Touch Response Switch

Method Signature:

<meta-data android:name="FORBID_TOUCH" android:value="true"/>

Method description:

After configuring this preprocessing instruction, the user will still be able to listen for touch events, but the robot will not respond to touch. The event will respond in any way, including default actions such as touching to wake up the robot.

3.11.5 PIR Response Switch

Method Signature:

<meta-data android:name="FORBID_PIR" android:value="true"/>

Method description:

After configuring this preprocessing instruction, the user can still listen for PIR trigger events, but the robot will not execute the PIR trigger events according to the default strategy (the default strategy means that when the robot hears that the PIR on the back of the robot has been triggered, the robot will rotate 180 degrees).

3.11.6 Voice wake-up response switch

method signature:

<meta-data android:name="FORBID_WAKE_RESPONSE" android:value="true"/>

Method description:

After configuring this preprocessing instruction, the robot will not perform any actions other than having its ear lights illuminate green when it is voice-activated. Any default response behavior.

3.11.7 Misidentified Pattern

Method Signature:

<meta-data

android:name="MISTAKENLY_IDENTIFIED"android:value="true"/>

Method description:

By default, the main controller filters the recognition of one character. After setting this preprocessing instruction, the main controller will no longer filter the recognition of any characters, such as "ÿ" (is), "ÿ" (not), etc. This mode is best used in a quiet environment; noisy environments will result in continuous misrecognition.

3.11.8 Anti-false positive mode (will be killed when playing music in the background)

method signature:

<meta-data android:name="forbid_stop_music" android:value="true"/>

Method description:

Previous versions would kill apps playing music in the background by default. For example, if music was playing on the current page and you navigated to another page while the music was still playing, the app would be killed. Now, a preprocessing mode has been added. Configuring this mode will prevent the app from being killed, but the app needs to manage the playback lifecycle itself to avoid affecting the recognition accuracy.

3.11.9 **Method signature** for disabling semantic request pattern (applicable only in international versions) :

<meta-data

android:name="forbid_request_semantics"android:value="1"/>

Method description:

Users configure this meta-data on the Activity corresponding to AndroidManifest. The main service reads this configuration. If the configuration is set to 1, the main service will not make network requests. Clients can make their own semantic requests based on the text recognition returned by the main service.

3.12 Command words and their semantics

3.12.1 Custom Global Command Terms

Users can control the robot using specific words or phrases, which we call command words. For example, we can use the command word "turn on the light" to control the robot to turn on the white light on its head, and use the command word "play a movie" to control the robot to open a movie app and play a movie. These command words are global and will always take effect regardless of the system interface.

The SDK supports developers in defining custom global command words to invoke a developer-specified Activity or Service. When using custom global command words, developers need to be aware of the following:

1. Global command words support offline recognition, and command words can still be effective even when there is no internet connection.

2. Each app supports 10 custom command words, and the entire system supports up to 100 custom global commands. Command words exceeding this limit will be automatically ignored.

3. The parsing order of command words is unpredictable, which means that if your app defines the same command words as other developers' apps, your command words may not take effect.

4. Global command words will be preloaded when the system starts for the first time. If the developer makes changes to the global command words during development, the robot or the main service program needs to be restarted for them to take effect.

The development process for custom global command words is as follows:

1> Add the following configuration to the AndroidManifest.xml file:

```
<provider
    android:authorities="<your app package name>.grammar"
    android:name="com.sanbot.opensdk.utils.GrammarProvider"
    android:exported="true"/>
```

Note that you should replace the content in the parentheses above with the corresponding app package name.

2> Create a new grammar file in the assets directory, and create a folder named [name missing] within the newly created grammar folder.

The file is globalgrammar.xml.

3> Write the syntax for global command words in globalgrammar.xml.

```
<Grammars>
    <group>
        <command>
            <item>Play video</item>
            <item>Open video</item>
        </command>
        <type>activity</type>
        <class>com.sanbot.librarydemo.VideoActivity</class>
    </group>
    <group>
        <command>
            <item>Start Service</item>
        </command>
        <type>service</type>
        <class>com.sanbot.librarydemo.MainService</class>
    </group>
</Grammars>
```

The above is an example of global command syntax. When a user says "play video" or "open video" to the bot, the bot will automatically open the com.sanbot.librarydemo.VideoActivity page. The following sections will provide a detailed explanation of the XML tags used in the example.

<Grammars> — The root tag of the syntax.

<group> — This defines a set of command words and the behavior of those command words to start an Activity or Service.

<command> — <command> contains a set of command words that have the same behavior.

<item> — Each <item> contains a command word.

<type> — The object launched when the command word is triggered; optional values are activity and service.

<class> — The class name of the class to be started. Note that the complete class name information must be filled in.

4. For the activity or service that needs to be started, add the following configuration to the androidmanifest.xml file.

Place:

android:exported="true"ÿ

```
<activity android:name=".VideoActivity"
    android:hardwareAccelerated="true"
    android:exported="true">
    <meta-data android:name="config_record" android:value="false"/>
</activity>
```

The triggered global command will be passed to the activity or service via an Intent. If the developer needs to obtain the trigger...

For command words, please refer to the following code.

```
public class TestActivity extends BindBaseActivity {

  @Override
  public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState); String
    data = getIntent().getStringExtra("content");
  }
  ……
}
```

3.12.2 Custom Semantics In

the speech recognition callback interface (refer to Section 3.2.6), the interface returns not only the user's speech to the robot but also the semantic parsing results that the robot can recognize. However, this semantic parsing support is imperfect and often cannot meet the semantic needs of developers in various specific scenarios. Therefore, the SDK provides methods for developers to customize semantics.

When using custom semantics, developers need to understand the following:

1. Custom semantics can generally only be used when connected to the internet.

2. Each app supports 20 custom semantic entries; semantic entries exceeding this limit will be automatically ignored.

3. Custom semantics only take effect when the user's app is in the foreground. Therefore, in the Service...

   Using custom semantics for listening is an unreliable practice.

4. Custom semantics will be preloaded when the system starts for the first time. If the developer makes changes to the custom semantics during development, the robot or the main service program needs to be restarted for them to take effect.

The development process for custom global command words is as follows:

1> Add the following configuration to the AndroidManifest.xml file:

```
<provider
  android:authorities="<your app package name>.grammar"
```

```
android:name="com.sanbot.opensdk.utils.GrammarProvider"
android:exported="true"/>
```

Please replace the content in parentheses above with the corresponding app package name.

2> Create a new grammar file in the assets directory, and then create a file named localgrammar.xml in the newly created grammar folder.

3> Write the syntax for global command words in localgrammar.xml.

```
<Grammars>
    <group>
        <command>
            <item>Play video</item>
            <item>Open video</item>
        </command>
        <topic>qh_video</topic>
        <action>start</action>
    </group>
    <group>
        <command>
            <item>Pause Video</item>
        </command>
        <topic>qh_video</topic>
        <action>pause</action>
    </group>
</Grammars>
```

The above is an example of global command syntax. When a user says "play video" or "open video" to the bot, the bot will automatically open the com.sanbot.librarydemo.VideoActivity page. The following sections will provide a detailed explanation of the XML tags used in the example.

<Grammars> — The root tag of the syntax.

<group> — A set of semantics is defined in <group>.

<command> — <command> contains a set of semantics that share the same topic and action.

<item> — Each <item> contains a phrase. When the robot recognizes this phrase, it will classify it according to the user's...

Custom semantic rules are used for processing.

<topic> — Defines the semantics of a topic. To avoid mixing with the system's built-in semantics, it is recommended to name it as "Company (Individual) Abbreviation_Topic Name". Please refer to the "Sanbao Open Semantic Specification Document" for details.

<action> — Defines the semantics of an action. For details, please refer to the "Sanbao Open Semantics Documentation".

3.13 Intelligent Peripheral Control

Access process:

1. Request a ZigbeeManager object. The request code is as follows:

```
1. ZigbeeManager zigbeeManager =
(ZigbeeManager)getUnitManager(FuncConstant.ZIGBEE_MANAGER); 2. Use the requested zigbeeManager object to call the
```

corresponding methods for control.

3.13.1 Obtaining the whitelist

method signature:

public OperationResult getWhiteList()

Method description:

This is used to retrieve the Zigbee whitelist. The actual result will be returned in the callback method.

3.13.2 Method signature for determining Zigbee readiness :

public OperationResult getNotifyReadyStatus()

Method description:

Check if Zigbee is ready. Other Zigbee operations can only be performed after Zigbee is ready.

Return value description:

The getResult() method of OperationResult will return the query result. If the returned value is "1", it means that Zigbee has been successfully initialized, and "0" means that Zigbee has not been successfully initialized.

Example code:

```
OperationResult operationResult =zigbeeManager.
getNotifyReadyStatus();
if(operationResult.getResult().equals("1")){
        //TODO NotifyReady
}
```

3.13.3 **Method signature for**

obtaining the device list:

public OperationResult getZigbeeList()

Method description:

Used to retrieve a list of Zigbee devices. The results will be returned in the callback method.

3.13.4 Adding to the whitelist

method signature:

public OperationResult addWhiteList(String command)

Method description:

Used to add Zigbee devices to the whitelist.

3.13.5 Signature of the method to

remove from the whitelist :

public OperationResult deleteWhiteList(String command)

Method description:

Used to remove Zigbee devices from the whitelist.

3.13.6 Enable whitelist

method signature:

public OperationResult switchWhtieList(boolean isOpen)

Method description:

Used to turn the whitelist function on or off.

Example code:

```
zigbeeManager.switchWhtieList(true);
```

3.13.7 Configure the allowed time method signatures for

device integration :

public OperationResult setAllowJoinTime(int time)

Method description:

This sets the time during which devices are allowed to join the gateway. When set to 0, it means that devices are allowed to join at any time.

Gateway.

Parameter description:

The time period allowed is in seconds.

Example code:

```
zigbeeManager.setAllowJoinTime(200);
```

3.13.8 Device Removal

Method Signature:

public OperationResult deleteDevice(String command)

Method description:

Remove the device from the gateway.

3.13.9 Signature for clearing

blank list **method:**

public OperationResult clearWhiteList()

Method description:

Clear the Zigbee whitelist.

3.13.10 Byte Sending Instruction

Method Signature:

public OperationResult sendByteCommand(byte[] data)

Method description:

Sending data of type byte array directly to the Zigbee module is generally used for communication between private devices.

3.13.11 Send String command

method signature:

public OperationResult sendCommand(String command)

Method description:

Send String type commands directly to the Zigbee module.

3.13.12 Zigbee Interface Callback

Method Signature:

public void setZigbeeListener(ZigbeeListener zigbeeListener)

Method description:

The Zigbee-related listening interface is used to obtain whitelist data and listen for Zigbee data updates and status changes.

Example code:

```
zigbeeManager.setZigbeeListener(new ZigbeeManager.ZigbeeListener()
{
  @Override
   public void notifyWhiteList(String whiteListData) {
      //Get whitelist data
   }

  @Override
   public void notifyStatusChange(String info) {
    //Device type data will be returned here.
   }

  @Override
   public void notifyInfo(String info) {
      //Device list data will be returned here.
```

```
    }
});
```

3.14 Face tracking and proactive greeting

Access process:

1. Request a FaceTrackManager object. The request code is as follows:

```
FaceTrackManager faceTrackManager =
(FaceTrackManager)getUnitManager(FuncConstant.FACETRACK_MANAGE R);
```

2. Use the obtained faceTrackManager object to call the corresponding methods for control.

3.14.1 Face tracking switch

method signature:

public void switchFaceTrack(boolean isOn)

Method description:

You can send a boolean command to **FaceTrackManager to enable** face tracking in non-face-based interfaces. Similarly, to disable face tracking, simply pass `false` (if you exit the interface without manually disabling face tracking, the system will automatically disable it when the client disconnects).

Example code:

```
faceTrackManager.switchFaceTrack(true);
```

3.14.2 Face tracking state callback

method signature:

public void setFaceTrackListener

(new FaceTrackManager.FaceTrackListener() {})

Method description:

The face tracking status callback will only occur if face tracking is enabled on the current page.

Example code:

```
faceTrackManager.setFaceTrackListener(new
FaceTrackManager.FaceTrackListener() {
    @Override
    public void faceTrackStart() {
        (Log! TAG, "faceTrackStart: ");
    }

    @Override
    public void faceTrackComplete() {
        (Log! TAG, "faceTrackComplete: ");
    }

    @Override
    public void faceTrackFail() {
        (Log! TAG, "faceTrackFail: ");
    }

    @Override
    public void faceTrackLost() {
        (Log! TAG, "faceTrackLost: ");
    }
```

```
    @Override
    public void startFindFace() {
        (Log! TAG, "startFindFace: ");
    }
});
```

1. The startFindFace() method triggers the ir function when a person stands in front of the robot. It will be called back if no person is detected. When this method is called back, it means that the robot is trying to find a face by moving.

2. The `faceTrackStart()` method indicates that a face has been detected and face tracking has begun. This method will only return one response per cycle. Adjust once

3. The faceTrackComplete() method indicates that the face has been tracked and is centered on the screen. Users can perform logical operations such as face recognition. Note that this method may be triggered multiple times, possibly after the user moves and the face is tracked back to the center.

4. Description of the `faceTrackLost()` method: If no face information is found within 10 seconds, the face has been lost. This method will only be triggered if... The `faceTrackStart` method will trigger the event.

5. The `faceTrackFail()` method is only triggered if no face is found within 10 seconds after the `startFindFace` method is executed. If a face is found within 10 seconds, this method will not be triggered.

3.14.3 **Signature for the method of**

initiating a greeting switch:

public void switchSayHello(boolean isOn)

Method description:

Send a boolean command directly to **FaceTrackManager. To use the active greeting feature,** please enable face tracking first, as the active greeting feature will only be triggered when the face is centered on the screen.

Example code:

```
faceTrackManager.switchSayHello(true);
```

`true` indicates enabled; `false` indicates disabled.

3.14.4 Callback for Initiating a Greeting

Method signature:

```
public void setUserNameDetectListener(new FaceTrackManager.UserNameDetectListener()
{})
```

Method description:

The system will save facial information for 5 seconds. If no facial information is saved within 5 seconds, the saved facial information will be automatically cleared. When the robot...

When not speaking, not being woken up, and not having the microphone muted, facial information will be recalled.

Example code:

```
faceTrackManager.setUserNameDetectListener(new
FaceTrackManager.UserNameDetectListener() {
    @Override
    public void userName(String s) {
        (Log! TAG, "userName: $s"+s);
    }
});
```

It should be noted that s may be an empty string.

3.15 Desktop Motion Control Protocol

Access process:

1. Request a DesktopMotionManager object. The request code is as follows:

```
DesktopMotionManager desktopManager =
 (DesktopMotionManager)getUnitManager(FuncConstant.DESKTOPMOTIO
N_MANAGER);
```

2. Use the obtained **desktopManager** object to call the corresponding methods for control.

3.15.1 Stop moving

Method signature:

```
public OperationResult doStop()
```

Method description:

The command is sent directly to the background service through **the desktopManager** object. Upon receiving the command, the background service will stop the multi-axis motion.

Example code:

```
desktopManager.doStop();
```

3.15.2 Desktop motion reset **method**

signature:

public OperationResult doReset()

Method description:

The command is sent directly to the background service through **the desktopManager** object. Upon receiving the command, the background service will perform a reset.

Example code:

```
desktopManager.doReset();
```

3.15.3 Signature of the method for relative angle

motion on the desktop :

public OperationResult

doRelativeAngleMotion(RelativeAngleDesktopMotion relativeAngleDesktopMotion)

Method description:

Commands are sent directly to the backend service through **the desktopManager** object. Upon receiving the command, the backend service will perform the corresponding operation.

Speed has positive and negative values, while angle does not.

Vertical angle control range: 0-45 degrees

Horizontal angle control range: 0-160°

Roll angle control range: 0-160

Forward and backward angle control range: 0-45

Example code:

```
RelativeAngleDesktopMotion relativeAngleDesktopMotion=new RelativeAngleDesktopMotion();
relativeAngleDesktopMotion.setForwardDegree((short) 10); // The required forward/backward motion angle is 10 degrees.
relativeAngleDesktopMotion.setForwardSpeed((short) 10); // Moves backward, speed control is 1-60, positive numbers move backward, negative numbers move forward, the speed is determined by positive and negative values
Degree control of forward and backward movement direction
relativeAngleDesktopMotion.setHorizontalDegree((short) 10); // Horizontal motion angle 10
relativeAngleDesktopMotion.setHorizontalTurnSpeed((short) 10); // Speed control: 1-60 moves left, positive numbers move left, negative numbers move right
The direction of movement is controlled by positive and negative velocities.
relativeAngleDesktopMotion.setVerticalDegree((short) 10); // Vertical up/down motion angle 10
relativeAngleDesktopMotion.setVerticalSpeed((short) 10); // Speed control 1-60. Moves upwards, positive numbers move upwards, negative numbers move downwards.
Speed controls the vertical movement direction
relativeAngleDesktopMotion.setRollDegree((short) 10); // Roll angle 10
relativeAngleDesktopMotion.setRollSpeed((short) 10); // Speed control 1-60. Rolls to the right, positive numbers move to the right, negative numbers move to the left.
Speed controls the left and right direction of roll.
desktopMotionManager.doRelativeAngleMotion(relativeAngleDesktopMotion);
```

3.15.4 Desktop Absolute Angle Motion

Method Signature:

public OperationResult

doAbsoluteAngleMotion(AbsoluteAngleDesktopMotion absoluteAngleDesktopMotion)

Method description:

Commands are sent directly to the backend service through **the desktopManager** object. Upon receiving the command, the backend service will perform the corresponding operation.

Absolute angle control; speed can only be a positive number.

Vertical angle control range: 0-45 degrees

Horizontal angle control range: 0-160°

Roll angle control range: 0-160

Forward and backward angle control range: 0-45

Example code:

```
AbsoluteAngleDesktopMotion absoluteAngleDesktopMotion=new
AbsoluteAngleDesktopMotion();
    absoluteAngleDesktopMotion.setForwardDegree((short) 10); // The required forward/backward motion angle is 10.
```

```
        absoluteAngleDesktopMotion.setForwardSpeed((short) 10);
            absoluteAngleDesktopMotion.setHorizontalDegree((short) 10); // Horizontal motion angle 10
        absoluteAngleDesktopMotion.setHorizontalTurnSpeed((short) 10);
            absoluteAngleDesktopMotion.setVerticalDegree((short) 10); // Vertical up/down motion angle 10
        absoluteAngleDesktopMotion.setVerticalSpeed((short) 10);
            absoluteAngleDesktopMotion.setRollDegree((short) 10); // Roll angle 10
        absoluteAngleDesktopMotion.setRollSpeed((short) 10);
desktopMotionManager.doAbsoluteAngleMotion(absoluteAngleDesktopMotion);
```